

Plateforme numérique de suivi fiscal et comptable

Centre de Gestion Agréé Broad Range Consulting Group

*Architecture logicielle, choix technologiques, modèle de données,
conventions d'implémentation et conditions d'exploitation*

Maître d'ouvrage

Broad Range Consulting Group

Représenté par Madame HIMSTA Paule Diane

Maîtrise d'œuvre

Monsieur TCHAMBA TCHAKOUNTE Edwin

Monsieur TONBA DJIMGOU Loic

Version 1.0 · Douala, le 8 août 2026

Document confidentiel

Sommaire

Objet et portée	4
1. Cohérence du socle technologique retenu	5
1.1 Analyse de la combinaison proposée	5
1.2 Deux réserves à porter à la connaissance du maître d'ouvrage	5
1.3 Technologies écartées et motifs	6
2. Vue d'ensemble de l'architecture	7
2.1 Composants et rôles	7
2.2 Flux principaux	8
2.3 Principes directeurs	8
3. Backend, organisation et conventions	10
3.1 Pile logicielle	10
3.2 Organisation du dépôt backend	11
3.3 Conventions de l'interface de programmation	11
3.4 Traitements différés et planifiés	12
4. Sécurité applicative	13
4.1 Authentification	13
4.2 Autorisation	13
4.3 Protections transverses	13
5. Modèle de données et invariants	15
5.1 Choix structurants	15
5.2 Entités principales par module	15
5.3 Cloisonnement par locataire	15
5.4 Référentiel temporel	16
5.5 Représentation des montants	16
5.6 Immuabilité comptable	17
5.7 Journal d'audit chaîné	17
5.8 Numérotation sans rupture	18
5.9 Indexation	18
6. Moteur de règles de conformité	19
6.1 Exigences de conception	19
6.2 Représentation d'une règle	19
6.3 Langage d'expression	20
6.4 Document projeté	21
6.5 Déroulement de l'évaluation	21
6.6 Chaînage vers l'aval	21
6.7 Administration et essai	22
7. Application web	23
7.1 Pile logicielle	23
7.2 Organisation et principes	23
7.3 États de dégradation	24
8. Application mobile	25
8.1 Périmètre et pile logicielle	25
8.2 Fonctionnement hors ligne	25
8.3 Contraintes d'exploitation mobile	26
9. Contrats d'interface et génération de clients	27
10. Stockage documentaire et traitement des pièces	28
10.1 Stockage	28
10.2 Ingestion	28
10.3 Reconnaissance de caractères	28
11. Intégrations externes	29
11.1 Principe d'abstraction	29
11.2 Intégrations prévues	29
12. Exploitation	30

12.1 Environnements	30
12.2 Hébergement	30
12.3 Sauvegarde et continuité	30
12.4 Supervision	31
13. Qualité et intégration continue	32
13.1 Organisation des dépôts	32
13.2 Discipline de contribution	32
13.3 Chaîne d'intégration	32
13.4 Contrôles spécifiques automatisés	32
13.5 Stratégie de tests	33
14. Performance et dimensionnement	34
14.1 Objectifs	34
14.2 Hypothèses de volumétrie	34
14.3 Règles de conception liées à la performance	34
15. Décisions d'architecture	35
16. Risques techniques	36
17. Validation	37
Annexe. Correspondance entre cas d'utilisation et composants	38

Objet et portée

Nature du document	Cahier des charges technique, complément du cahier des charges fonctionnel version 1.0 du 8 août 2026
Version et date	Version 1.0 du 8 août 2026
Auteurs	Monsieur TCHAMBA TCHAKOUNTE Edwin et Monsieur TONBA DJIMGOU Loic
Destinataires	Madame HIMSTA Paule Diane pour information et validation des engagements d'exploitation, équipe de réalisation pour exécution
Socle technologique	Backend Python et FastAPI, application web Next.js, application mobile Flutter, base de données PostgreSQL

Le présent document définit l'architecture technique de la plateforme, les technologies retenues, les conventions d'implémentation et les conditions d'exploitation. Il traduit en décisions d'ingénierie les exigences énoncées dans le cahier des charges fonctionnel, auquel il se réfère par les identifiants de cas d'utilisation UC01 à UC11 et de domaines fonctionnels F1 à F11.

Il s'adresse à deux lectorats. Les chapitres 1, 2, 14 et 16 sont destinés au maître d'ouvrage et exposent les choix structurants, leurs justifications et leurs implications en matière de coût, de délai et d'exploitation. Les chapitres 3 à 13 sont destinés à l'équipe de réalisation et constituent la référence normative des développements.

Les prescriptions y sont formulées selon la même échelle que dans le document fonctionnel. Une prescription introduite par *doit* est impérative et son non-respect constitue un défaut de conformité. Une prescription introduite par *devrait* admet une dérogation motivée et consignée. Une prescription introduite par *pourra* relève de l'évolution.

1. Cohérence du socle technologique retenu

1.1 Analyse de la combinaison proposée

Le socle retenu associe un backend en Python avec FastAPI, une application web en Next.js et une application mobile en Flutter. Cette combinaison est cohérente et adaptée au problème posé, pour quatre raisons.

La première tient au contrat d'interface. FastAPI produit automatiquement une description OpenAPI complète et typée de son interface, à partir des modèles Pydantic qui valident déjà les entrées et sorties. Cette description devient la source unique de vérité à partir de laquelle sont générés le client TypeScript consommé par Next.js et le client Dart consommé par Flutter. Trois bases de code, mais un seul contrat, vérifié automatiquement à chaque intégration continue. C'est le principal argument de cohérence de l'ensemble.

La deuxième tient à la nature du travail des deux interfaces, qui diffère profondément. Le poste du collaborateur du Centre appelle des écrans denses, des tableaux volumineux, de la saisie rapide au clavier et de l'impression, ce que le web sert nettement mieux qu'une application native. L'adhérent, lui, a besoin de photographier une facture au moment où il la reçoit, souvent sans réseau. Ce besoin appelle un accès natif à l'appareil photo, un stockage local robuste et une synchronisation différée, ce que Flutter fournit avec une base de code unique pour Android et iOS. Attribuer chaque technologie à l'usage qui la justifie est ici un choix juste.

La troisième tient à l'écosystème Python pour le domaine traité. La manipulation de données comptables, la génération de la liasse fiscale au format tabulaire attendu par l'administration, le traitement documentaire et la reconnaissance de caractères disposent en Python de bibliothèques matures. Le moteur de règles décrit au chapitre 6 se prête particulièrement bien à une implémentation en Python.

La quatrième tient à la disponibilité des compétences. Python, TypeScript et Dart sont enseignés et pratiqués localement, ce qui préserve la capacité du maître d'ouvrage à faire reprendre la maintenance par un tiers, exigence énoncée au chapitre des orientations techniques du document fonctionnel.

1.2 Deux réserves à porter à la connaissance du maître d'ouvrage

Première réserve, relative à la charge. Trois bases de code représentent, pour une équipe de deux personnes, un effort de maintenance permanent qu'il serait imprudent d'engager simultanément.

La maîtrise d'œuvre recommande en conséquence de différer l'application Flutter à la phase 3 du planning, celle qui porte la collecte des pièces et le moteur de conformité, et de la limiter à son périmètre justifié, à savoir la capture, la consultation des échéances et le suivi du dossier. L'ensemble des fonctions de gestion demeure sur le web, l'application Next.js étant conçue dès l'origine pour un usage confortable sur écran mobile. Cette progression permet de disposer d'un produit utile dès la phase 2 sans immobiliser l'équipe sur trois fronts.

Seconde réserve, relative à la représentation des montants. Python manipule par défaut des nombres à virgule flottante, dont l'usage en comptabilité produit des écarts de centimes qui invalident un rapprochement et compromettent une liasse.

La règle énoncée au point 5.5 est en conséquence impérative et non négociable. Aucun montant ne transite ni n'est stocké sous forme de nombre à virgule flottante, à aucun endroit de la chaîne, du formulaire Next.js jusqu'à la colonne PostgreSQL, en passant par les modèles Pydantic et les objets Dart. Un contrôle automatisé décrit au point 13.4 vérifie cette règle à chaque intégration.

1.3 Technologies écartées et motifs

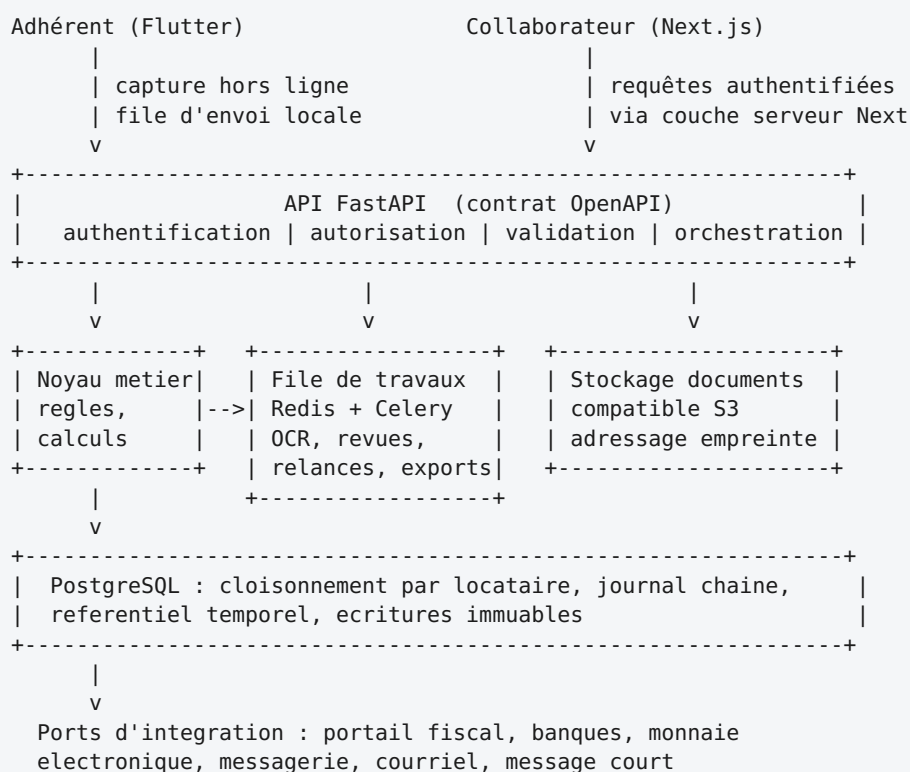
Option écartée	Motif
Architecture en services indépendants	Complexité opérationnelle disproportionnée pour une équipe de deux personnes. Les transactions comptables, qui doivent être atomiques d'un bout à l'autre, s'accommodent mal d'une répartition. Un découpage modulaire interne procure la même discipline sans le coût.
Cadre applicatif intégré de type Django	L'interface d'administration automatique, principal argument, est inutilisable ici puisque toute écriture doit passer par les invariants métier. Le typage strict et la génération de contrat d'interface de FastAPI servent mieux le besoin.
Base de données documentaire	La comptabilité est un domaine relationnel par nature, exigeant intégrité référentielle, transactions et agrégations. Une base documentaire imposerait de reconstruire ces garanties en applicatif.
Développement mobile natif séparé	Doublerait la charge de développement mobile sans bénéfice, le besoin ne comportant aucune fonction dépendant d'un système d'exploitation particulier.
Bibliothèque de règles métier générique	Les moteurs génériques disponibles imposent leur formalisme, se prêtent mal à la traçabilité du fondement légal exigée au point 6.2 et sont difficiles à administrer par un non-informaticien. Un moteur dédié, décrit au chapitre 6, est plus adapté et plus auditable.

2. Vue d'ensemble de l'architecture

2.1 Composants et rôles

Composant	Technologie	Rôle
Interface web	Next.js, TypeScript	Poste de travail des collaborateurs du Centre et portail des adhérents
Application mobile	Flutter, Dart	Capture des pièces hors ligne, consultation des échéances par l'adhérent
Interface de programmation	FastAPI, Python	Point d'entrée unique, validation, autorisation, orchestration des cas d'utilisation
Noyau métier	Python	Règles de gestion, moteur de conformité, calculs fiscaux, production de la liasse
Base de données	PostgreSQL	Persistance transactionnelle, cloisonnement par locataire, journal d'audit
Files et cache	Redis	Support des traitements différés, cache des paramètres et des vérifications externes
Travailleurs asynchrones	Celery	Reconnaissance de caractères, revues périodiques, relances, génération de documents
Ordonnanceur	Celery beat	Déclenchement des traitements planifiés, échéancier, relances, contrôles de masse
Stockage documentaire	Service compatible S3	Conservation des pièces justificatives sur dix ans, adressage par empreinte
Supervision	Journalisation structurée, métriques, collecte d'erreurs	Exploitation, diagnostic, suivi du niveau de service

2.2 Flux principaux



2.3 Principes directeurs

Sept principes gouvernent l'ensemble des développements. Ils sont opposables en revue de code et lors de la recette technique.

Principe	Énoncé
Norme comme donnée	Aucune valeur légale, taux, seuil, barème ou date d'échéance n'apparaît dans le code. Toute norme réside en base, portée par une période de validité, et se lit à une date donnée.
Immuabilité comptable	Une donnée comptable validée n'est jamais modifiée ni supprimée. La correction procède d'une écriture nouvelle. La contrainte est appliquée par la base et non par l'applicatif seul.
Traçabilité intégrale	Toute opération significative est consignée dans un journal chaîné par empreinte, rendant toute altération détectable.
Cloisonnement par défaut	L'isolation des données entre locataires et entre portefeuilles est garantie au niveau de la base, non par la vigilance du développeur.
Dépendances inversées	Le noyau métier ne dépend d'aucune technologie externe. Les systèmes tiers sont atteints au travers d'interfaces abstraites, chacune disposant d'un mode de substitution manuel.
Contrat unique	La description OpenAPI produite par le backend est la seule source de vérité des interfaces. Les clients web et mobile en dérivent par génération.

Exactitude numérique	Aucun montant n'emprunte de représentation à virgule flottante, en aucun point de la chaîne.
----------------------	--

3. Backend, organisation et conventions

3.1 Pile logicielle

Élément	Retenu	Justification
Langage	Python 3.12	Typage progressif mature, écosystème adapté au domaine
Cadre web	FastAPI	Validation déclarative, génération du contrat OpenAPI, performances suffisantes
Validation et sérialisation	Pydantic v2	Typage strict des entrées et sorties, prise en charge native des décimaux
Accès aux données	SQLAlchemy 2.0, style typé	Contrôle fin des transactions, indispensable pour les invariants comptables
Pilote base	psycopg 3	Prise en charge complète des types PostgreSQL, dont numeric et jsonb
Migrations	Alembic	Versionnement du schéma, réversibilité
Travaux différés	Celery, Redis	Maturité, documentation abondante, ordonnancement intégré
Qualité	ruff, mypy en mode strict, pytest	Détection précoce, contrat de typage sur le noyau métier

3.1.1 Mode d'exécution synchrone

Les points d'entrée doivent être déclarés en mode synchrone, FastAPI les exécutant alors dans un réservoir de fils d'exécution. Ce choix, contraire à l'usage courant, se justifie ici. Les traitements comptables sont dominés par des transactions de base de données de courte durée, non par de l'attente réseau. L'écriture asynchrone imposerait une vigilance permanente sur la portée des transactions et sur les appels bloquants involontaires, source d'erreurs subtiles et coûteuses à diagnostiquer. Le gain de débit serait, à la charge attendue, imperceptible.

Les rares traitements dominés par l'attente réseau, notamment les appels aux systèmes tiers, doivent être confiés aux travailleurs asynchrones plutôt que traités dans le fil de la requête.

3.2 Organisation du dépôt backend

backend/	
app/	
core/	configuration, securite, journalisation, erreurs
config.py	parametres d'environnement valides par Pydantic
security.py	mots de passe, jetons, second facteur
db.py	moteur, session, pose du contexte de locataire
audit.py	écriture du journal chaine
errors.py	hierarchie d'exceptions et correspondance HTTP
shared/	types et utilitaires partages, sans metier
money.py	type Montant, arrondis, conversions
ids.py	generation d'identifiants ordonnes
period.py	intervalles de validite, chevauchements
modules/	
referentiel/	F1 parametres, baremes, plan comptable
portefeuille/	F2 entreprises, regimes, exercices, adhesion
pieces/	F3 collecte, cycle de vie, rapprochement
conformite/	F4 moteur de regles, rapports, consequences
comptabilite/	F5 journaux, ecritures, balances, lettrage
obligations/	F6 echeancier, declarations, paiements
cloture/	F7 inventaire, liasse, tableau de passage
formalites/	F8 creation d'entreprise, conversion
pilotage/	F9 indicateurs et tableaux de bord
identite/	F11 comptes, roles, habilitations
integrations/	ports et adaptateurs vers les systemes tiers
workers/	taches Celery et ordonnancement
main.py	
migrations/	revisions Alembic
tests/	
pyproject.toml	

Chaque module doit présenter la même structure interne, à savoir un fichier de points d'entrée, un dossier de schémas d'échange, un dossier de modèles de persistance, un dossier de services applicatifs et un dossier de domaine contenant les règles pures. Le domaine ne doit importer ni SQLAlchemy, ni FastAPI, ni aucun module d'infrastructure. Cette règle est vérifiée automatiquement par un test d'architecture décrit au point 13.4.

Les dépendances entre modules doivent être explicites et orientées. Un module ne peut appeler un autre module que par son service applicatif, jamais par ses modèles de persistance. Les dépendances circulaires sont proscrites et détectées automatiquement.

3.3 Conventions de l'interface de programmation

Sujet	Convention
Préfixe et version	Toutes les ressources sous <code>/api/v1</code> . Un changement incompatible impose une version nouvelle, les deux versions coexistant le temps de la migration des clients.
Nommage	Ressources au pluriel en français, identifiants en chemin, filtres en paramètres de requête.
Pagination	Pagination par curseur opaque, jamais par décalage numérique, afin de rester stable sur les collections volumineuses et en croissance.

Erreurs	Corps normalisé selon le format Problem Details, comportant un code d'erreur applicatif stable, un message destiné à l'utilisateur et, pour les erreurs de validation, le détail par champ.
Idempotence	Toute opération de création accepte un en-tête de clé d'idempotence. Une clé déjà traitée retourne la réponse initiale sans réexécution. Obligatoire pour la synchronisation mobile.
Concurrence	Les modifications utilisent le contrôle optimiste par version de ressource, un conflit produisant une réponse explicite plutôt qu'un écrasement silencieux.
Montants	Sérialisés en chaîne de caractères décimale, jamais en nombre, afin d'éviter toute conversion en virgule flottante par les clients.
Dates	Dates comptables en date pure sans fuseau. Horodatages techniques en temps universel coordonné.
Corrélation	Chaque requête porte un identifiant de corrélation, propagé aux travaux différés et présent dans toutes les traces.

3.4 Traitements différés et planifiés

Traitement	Déclenchement	Cas couvert
Reconnaissance de caractères sur pièce déposée	Événement de dépôt	UC03
Contrôle de conformité en masse	Planifié, mensuel et annuel	UC05
Détection des pièces manquantes et relances	Planifié, quotidien	UC04
Génération et recalcul de l'échéancier	Événement et planifié	UC08
Rappels d'échéance gradués	Planifié, quotidien	UC08
Détection de dépassement de seuil de régime	Planifié, mensuel	UC08
Génération des états et de la liasse	À la demande	UC10
Vérification des identifiants fiscaux	Événement, avec cache	UC05
Sauvegarde et vérification d'intégrité du journal	Planifié, quotidien	Transverse

Toute tâche doit être idempotente, une réexécution ne devant produire aucun effet supplémentaire. Toute tâche doit porter l'identifiant de locataire et l'identifiant de corrélation. Les échecs doivent être réessayés selon un délai croissant, puis, après épuisement, déposés dans une file d'échecs faisant l'objet d'une supervision. Aucune tâche ne doit s'exécuter sans limite de durée.

4. Sécurité applicative

4.1 Authentification

Les mots de passe doivent être dérivés par Argon2id, jamais stockés en clair ni sous une fonction de hachage rapide. La politique de robustesse doit être paramétrable et vérifiée à l'inscription comme au changement.

L'authentification produit un jeton d'accès de courte durée, quinze minutes, et un jeton de rafraîchissement de longue durée, révocable individuellement. La rotation du jeton de rafraîchissement doit être appliquée à chaque usage, et la réutilisation d'un jeton déjà consommé doit entraîner la révocation immédiate de toute la lignée, cette réutilisation signalant un vol de jeton.

Le second facteur, fondé sur un code temporel à usage unique, doit être obligatoire pour les profils de réviseur, de fiscaliste et d'administrateur. Des codes de secours à usage unique doivent être délivrés à l'activation.

4.1.1 Traitement des jetons côté web

L'application Next.js ne doit à aucun moment détenir de jeton dans le stockage du navigateur, cette pratique exposant à l'exfiltration par injection de script. Elle doit implémenter un intermédiaire côté serveur. Le jeton de rafraîchissement est déposé dans un témoin de connexion inaccessible au script, sécurisé et restreint au même site. Les appels du navigateur s'adressent aux gestionnaires de routes de Next.js, qui détiennent le jeton d'accès en mémoire côté serveur et relaient vers l'interface de programmation.

4.1.2 Traitement des jetons côté mobile

L'application Flutter doit conserver ses jetons dans le magasin sécurisé du système, à savoir le trousseau sur iOS et le magasin adossé au matériel sur Android. Le rafraîchissement doit être sérialisé, un verrou empêchant plusieurs requêtes concurrentes de déclencher des rotations simultanées.

4.2 Autorisation

Le contrôle d'accès repose sur trois niveaux qui se cumulent et ne se substituent pas.

Le premier niveau est le locataire. Toute session pose son identifiant de locataire dans le contexte de la connexion à la base, et la sécurité au niveau des lignes décrite au point 5.3 en assure le respect. Une erreur applicative ne peut donc pas provoquer de fuite entre cabinets.

Le deuxième niveau est le rôle, auquel sont rattachées des permissions élémentaires nommées selon la forme domaine et action. Les points d'entrée déclarent la permission requise, jamais le rôle, afin que la matrice reste modifiable sans reprise de code.

Le troisième niveau est le portefeuille. Un collaborateur n'accède qu'aux entreprises qui lui sont affectées, sauf permission d'accès étendu explicitement attribuée. Ce filtre est appliqué dans la couche d'accès aux données et non dans les points d'entrée, afin qu'aucun chemin ne puisse le contourner.

4.3 Protections transverses

Mesure	Prescription
--------	--------------

Limitation de débit	Par adresse et par compte, avec seuils resserrés sur l'authentification et le renvoi de code
Origines autorisées	Liste blanche stricte, aucune autorisation générale
Politique de contenu	Directive stricte sur l'application web, sans exécution de script en ligne
Injection	Requêtes exclusivement paramétrées, aucune composition de requête par concaténation
Téléversement	Contrôle du type réel par signature binaire, limite de taille, analyse antivirus avant mise à disposition
Secrets	Aucun secret dans le dépôt, injection par variables d'environnement, rotation documentée
Dépendances	Analyse automatique des vulnérabilités à chaque intégration, blocage sur vulnérabilité critique
Chiffrement	Transport chiffré obligatoire, chiffrement au repos de la base et du stockage documentaire
Référentiel de conformité	Niveau 2 du référentiel de vérification de sécurité applicative de l'OWASP visé avant mise en production

5. Modèle de données et invariants

5.1 Choix structurants

Sujet	Décision
Moteur	PostgreSQL 16 ou version supérieure
Identifiants	Identifiants universels ordonnés dans le temps, générables hors ligne par le client mobile sans risque de collision
Montants	Type numérique exact à dix-huit chiffres significatifs et deux décimales
Données semi-structurées	Colonnes jsonb réservées aux prédicats de règles et aux instantanés d'audit, jamais aux données métier interrogeables
Suppression	Aucune suppression physique sur les entités métier, marquage logique et horodatage
Horodatage	Colonnes de création et de modification systématiques, alimentées par la base

5.2 Entités principales par module

Module	Entités
Référentiel	parametre_fiscal, bareme, tranche_bareme, texte_legal, compte_reference, mapping_poste_liasse, type_obligation, regle_conformite, regle_version, regle_cas_test
Portefeuille	entreprise, etablissement, dirigeant, associe, regime_periode, rattachement_periode, exercice, adhesion, mandat_declaratif, affectation_collaborateur
Pièces	piece_justificative, fichier, extraction_ocr, demande_piece, relance, rapprochement_mouvement
Conformité	rapport_conformite, constat, consequence_fiscale, derogation
Comptabilité	journal, compte_entreprise, ecriture, ligne_ecriture, lettrage, releve_bancaire, mouvement_bancaire, immobilisation, plan_amortissement
Obligations	obligation_instance, declaration, ligne_declaration, paiement, penalite, accuse_depot
Clôture	travaux_cloture, balance_definitive, liasse, poste_liasse, tableau_passage, ligne_passage, controle_coherence
Formalités	prospect, dossier_creation, piece_dossier, etape_instruction, devis
Identité	locataire, utilisateur, role, permission, session, jeton_rafraichissement, second_facteur
Transverse	audit_log, sequence_compteur, tache_asynchrone, notification

5.3 Cloisonnement par locataire

Chaque table métier doit porter une colonne d'identifiant de locataire, non nulle et indexée. La sécurité au niveau des lignes doit être activée sur ces tables, avec une politique restreignant l'accès aux lignes dont l'identifiant correspond au paramètre de session. Le rôle applicatif ne doit pas disposer de l'attribut permettant de contourner ces politiques.

```

ALTER TABLE ecriture ENABLE ROW LEVEL SECURITY;
ALTER TABLE ecriture FORCE ROW LEVEL SECURITY;

CREATE POLICY p_ecriture_locataire ON ecriture
  USING (tenant_id = current_setting('app.tenant_id')::uuid)
  WITH CHECK (tenant_id = current_setting('app.tenant_id')::uuid);

```

La pose du paramètre de session doit être effectuée à l'ouverture de chaque transaction par un intercepteur de la couche d'accès aux données, à partir du contexte d'authentification. Un test automatisé doit vérifier qu'aucune requête ne s'exécute sans ce paramètre, et qu'une session posée sur un locataire ne peut lire aucune ligne d'un autre.

5.4 Référentiel temporel

Toute donnée de référence porteuse d'une norme doit être versionnée dans le temps par un intervalle de validité, la borne supérieure demeurant ouverte pour la version en vigueur. La lecture s'effectue impérativement à une date fournie par l'appelant.

```

CREATE TABLE parametre_fiscal (
  id          uuid PRIMARY KEY,
  tenant_id   uuid NOT NULL,
  code        text NOT NULL,
  valeur      numeric(18,6) NOT NULL,
  unite       text NOT NULL,
  valide_du   date NOT NULL,
  valide_au   date,
  texte_legal_id uuid NOT NULL REFERENCES texte_legal(id),
  cree_par    uuid NOT NULL,
  cree_le     timestampz NOT NULL DEFAULT now(),
  CONSTRAINT ck_periode CHECK (valide_au IS NULL OR valide_au > valide_du),
  EXCLUDE USING gist (
    tenant_id WITH =, code WITH =,
    daterange(valide_du, valide_au, '[') WITH &&
  )
);

```

La contrainte d'exclusion interdit tout recouvrement de périodes pour un même code, garantissant qu'à toute date une seule valeur s'applique. La référence au texte légal est obligatoire, conformément au principe énoncé au point 2.3.

Un service unique doit centraliser la lecture, sous la forme d'un appel prenant le code et la date. Aucun accès direct à la table n'est autorisé depuis les modules métier. Le service doit conserver un cache, invalidé à toute écriture, dont la clé comprend la date afin qu'un retraitement d'exercice antérieur reste exact.

5.5 Représentation des montants

Prescription impérative. Les montants sont manipulés exclusivement en décimal exact, de la saisie jusqu'au stockage.

En base, les colonnes monétaires sont de type numérique exact. En Python, les montants sont portés par le type décimal, jamais par le type flottant, avec un contexte d'arrondi fixé globalement au demi supérieur. Dans les schémas d'échange, les montants sont déclarés en décimal et sérialisés en chaîne de caractères. En

TypeScript, ils sont manipulés par une bibliothèque de décimaux et jamais par le type numérique natif. En Dart, ils sont portés par un type décimal dédié.

Le franc de la coopération financière en Afrique centrale ne comportant pas de subdivision en usage, les montants exigibles doivent être arrondis à l'unité au moment de la liquidation, les calculs intermédiaires conservant deux décimales. La règle d'arrondi, l'unité de conversion et le niveau auquel il s'applique, à la ligne ou au pied de document, doivent être des paramètres du référentiel et non des constantes du code, les pratiques pouvant varier.

5.6 Immuabilité comptable

L'immuabilité ne doit pas reposer sur la seule discipline applicative. Elle doit être imposée par la base au moyen d'un déclencheur interdisant toute modification ou suppression d'une écriture validée.

```
CREATE OR REPLACE FUNCTION f_ecriture_immuable() RETURNS trigger AS $$
BEGIN
    IF OLD.statut = 'VALIDEE' THEN
        RAISE EXCEPTION
            'Ecriture % validee : modification et suppression interdites', OLD.id
            USING ERRCODE = 'restrict_violation';
    END IF;
    RETURN CASE WHEN TG_OP = 'DELETE' THEN OLD ELSE NEW END;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER tg_ecriture_immuable
BEFORE UPDATE OR DELETE ON ecriture
FOR EACH ROW EXECUTE FUNCTION f_ecriture_immuable();
```

La correction d'une écriture validée s'effectue par contre-passation. Le service correspondant doit créer une écriture de sens inverse, la relier à l'écriture d'origine et exiger un motif obligatoire, l'ensemble étant consigné au journal d'audit.

5.7 Journal d'audit chaîné

Le journal d'audit doit être strictement en ajout, protégé par le même mécanisme de déclencheur. Chaque enregistrement doit porter l'empreinte de l'enregistrement précédent du même locataire, de sorte que la suppression ou la modification d'une entrée intermédiaire rompe la chaîne et devienne détectable.

```
CREATE TABLE audit_log (
    id                uuid PRIMARY KEY,
    tenant_id         uuid NOT NULL,
    sequence          bigserial NOT NULL,
    horodatage        timestamptz NOT NULL DEFAULT now(),
    utilisateur       uuid,
    correlation       uuid,
    entite            text NOT NULL,
    entite_id         uuid,
    action            text NOT NULL,
    avant             jsonb,
    apres             jsonb,
    empreinte_precedente bytea,
    empreinte         bytea NOT NULL
);
```

L'empreinte est calculée sur la concaténation de l'empreinte précédente et de la représentation canonique de l'entrée. Une tâche planifiée quotidienne doit parcourir la chaîne et alerter immédiatement en cas de rupture. Ce dispositif constitue l'élément probant que le Centre peut opposer en cas de contrôle, conformément à l'exigence de traçabilité du document fonctionnel.

5.8 Numérotation sans rupture

La continuité de la numérotation des factures et des écritures est une exigence légale rappelée au chapitre des contrôles. Les séquences natives de la base ne conviennent pas, une transaction annulée y créant un trou définitif.

L'attribution doit reposer sur une table de compteurs verrouillée, l'incrément et l'insertion s'effectuant dans la même transaction que la création du document, de sorte qu'une annulation restitue le numéro.

```
-- au sein de la transaction de creation du document
SELECT prochain FROM sequence_compteur
WHERE tenant_id = :t AND entreprise_id = :e
      AND domaine = :d AND exercice_id = :x
FOR UPDATE;

UPDATE sequence_compteur SET prochain = prochain + 1
WHERE tenant_id = :t AND entreprise_id = :e
      AND domaine = :d AND exercice_id = :x;
```

Le verrou étant pris sur une ligne unique par entreprise, par domaine et par exercice, la sérialisation qu'il impose demeure sans effet perceptible sur les performances. Un contrôle de détection des ruptures doit néanmoins être exécuté lors des revues périodiques, une rupture pouvant provenir de documents établis hors du système.

5.9 Indexation

Les index suivants doivent être créés dès l'origine, les schémas d'accès étant connus. Tout index supplémentaire doit être justifié par un plan d'exécution consigné.

- Index composé sur locataire, entreprise et exercice pour les écritures et leurs lignes, socle de tous les états comptables.
- Index composé sur locataire, entreprise et période pour les pièces justificatives et les obligations.
- Index sur compte et date pour les lignes d'écriture, servant le grand livre et la balance.
- Index sur numéro d'identifiant fiscal pour les tiers, servant les contrôles de conformité.
- Index sur date d'échéance et statut pour les obligations, servant l'échéancier consolidé et les relances.
- Index sur locataire et séquence pour le journal d'audit, servant la vérification de chaîne.

Le partitionnement des lignes d'écriture par exercice devra être envisagé lorsque le volume dépassera dix millions de lignes. Il n'est pas mis en œuvre initialement, la complexité qu'il introduit n'étant pas justifiée à la volumétrie attendue.

6. Moteur de règles de conformité

Le moteur de conformité constitue le composant le plus spécifique de la plateforme et celui dont dépend l'essentiel de la valeur métier. Sa spécification technique est en conséquence développée ici avec un niveau de détail supérieur.

6.1 Exigences de conception

Le moteur doit satisfaire simultanément cinq exigences dont la conjonction écarte l'usage d'une bibliothèque générique. Il doit être administrable par le fiscaliste sans intervention de développement. Il doit être versionné dans le temps, une opération étant évaluée selon les règles applicables à sa date. Il doit produire un résultat explicable, chaque constat renvoyant à son fondement légal. Il doit être déterministe et reproductible, l'évaluation d'un même document dans un même contexte donnant toujours le même résultat. Il doit enfin être sûr, l'exécution d'une expression saisie par un utilisateur ne devant en aucun cas permettre l'exécution de code arbitraire.

6.2 Représentation d'une règle

Une règle est persistée dans une table dédiée, ses versions successives dans une table liée. Le prédicat est stocké sous forme d'arbre d'expression en notation JSON, jamais sous forme de code source. Les attributs de fondement légal, de gravité et de conséquence fiscale sont obligatoires, l'activation étant refusée en leur absence.

```

{
  "code": "FAC-ACH-007",
  "libelle": "Reglement en especes au-dela du seuil legal",
  "categorie": "DEDUCTIBILITE",
  "texte_legal_id": "...",
  "valide_du": "2026-01-01",
  "valide_au": null,
  "portee": {
    "types_document": ["FACTURE_ACHAT"],
    "regimes_emetteur": ["REEL"],
    "exclusions": ["FOURNISSEUR_ETRANGER"]
  },
  "gravite": "MAJEUR",
  "predicat": {
    "op": "not",
    "args": [{
      "op": "and",
      "args": [
        { "op": "gte", "args": [
          { "var": "document.montant_ttc" },
          { "param": "SEUIL_ESPECES_TVA" } ] },
        { "op": "eq", "args": [
          { "var": "document.mode_reglement" }, "ESPECES" ] }
      ]
    }
  ]
},
  "consequence": {
    "tva_deductible": false,
    "charge_deductible": true,
    "poste_reintegration": null
  },
  "message": "Reglement en especes au-dela du seuil : TVA non deductible.",
  "remediation": "Justifier un reglement par voie bancaire ou electronique."
}

```

6.3 Langage d'expression

Le langage doit être volontairement restreint. Il ne comporte ni boucle, ni affectation, ni définition de fonction, ni accès à l'environnement d'exécution. Il se limite aux opérateurs et fonctions figurant dans une liste blanche.

Catégorie	Éléments admis
Logique	et, ou, non
Comparaison	égal, différent, supérieur, supérieur ou égal, inférieur, inférieur ou égal
Ensemble	appartenance, non appartenance
Texte	expression rationnelle, longueur, vide ou non renseigné
Numérique	somme, valeur absolue, arrondi, écart relatif
Accès	variable du document projeté, paramètre du référentiel lu à la date de l'opération
Fonctions métier	validité formelle d'un identifiant fiscal, activité d'un identifiant fiscal, régime d'un tiers à une date, existence d'un doublon, présence d'une rupture de séquence

Les fonctions métier doivent être injectées dans le contexte d'évaluation sous forme de fonctions pures, leurs éventuels appels externes ayant été résolus en amont. L'évaluateur ne doit effectuer aucun accès réseau ni aucune lecture en base, cette contrainte garantissant à la fois le déterminisme et la possibilité de tester chaque règle isolément.

6.4 Document projeté

Les règles ne doivent jamais opérer sur les modèles de persistance, dont la structure évoluera. Elles opèrent sur une projection stable et versionnée du document, construite par un service dédié. Cette projection agrège les données du document, celles de l'émetteur et du destinataire à la date de l'opération, les paramètres nécessaires et les résultats des vérifications externes déjà obtenues.

La projection doit être conservée avec le rapport de conformité. Cette conservation est indispensable pour rejouer un contrôle à l'identique plusieurs années après, lors d'un contrôle fiscal, sans dépendre de l'état courant de la base.

6.5 Déroulement de l'évaluation

1. Construction de la projection du document a la date de l'operation
2. Chargement du jeu de regles valide a cette date, depuis le cache
3. Filtrage des regles par portee : type de document, regime, exclusions
4. Evaluation independante de chaque regle retenue
5. Pour chaque predicat non satisfait, production d'un constat
6. Agregation des consequences fiscales des constats
7. Persistance du rapport, de la projection et des constats
8. Application des consequences aux attributs de la piece
9. Emission d'un evenement de conformite consomme par les modules aval

L'évaluation d'une règle ne doit jamais interrompre l'évaluation des autres. Une erreur technique survenant sur une règle doit produire un constat de nature technique, signalé au fiscaliste, sans compromettre le rapport.

L'agrégation des conséquences doit appliquer la règle du plus contraignant. Si une règle refuse la déduction de la taxe et une autre l'admet, la déduction est refusée.

6.6 Chaînage vers l'aval

Le rapport de conformité ne constitue pas une fin en soi. Ses conséquences doivent être matérialisées sur les données comptables, faute de quoi le dispositif resterait déclaratif.

Les conséquences sont portées par des attributs des lignes d'écriture, alimentés lors de la comptabilisation décrite au cas UC07 à partir du rapport rattaché à la pièce. La détermination de la taxe déductible de la période, au cas UC09, exclut les montants portant un refus de déduction. La constitution du tableau de passage, au cas UC10, agrège les montants portant une réintégration, en les regroupant par poste. Le chemin allant du constat de conformité jusqu'à la ligne du tableau de passage doit être reconstituable dans les deux sens, cette réversibilité faisant l'objet d'un critère d'acceptation.

6.7 Administration et essai

L'interface d'administration destinée au fiscaliste doit permettre la composition du prédicat par une construction guidée, sans saisie de texte technique, et proposer en regard une lecture en langage naturel de l'expression composée.

Une règle nouvellement créée ou modifiée doit être en état d'essai. Dans cet état, elle est évaluée sur les documents mais ses constats sont enregistrés sans effet, ce qui permet de mesurer son incidence sur un échantillon réel avant activation. L'activation requiert une confirmation explicite et est consignée au journal d'audit.

Chaque règle doit être associée à au moins deux cas de test persistés, l'un satisfaisant le prédicat, l'autre non. Ces cas sont exécutés à chaque intégration continue et à chaque activation. Une règle dont les cas de test échouent ne peut être activée.

7. Application web

7.1 Pile logicielle

Élément	Retenu	Rôle
Cadre applicatif	Next.js, routeur applicatif	Rendu côté serveur des pages de consultation, intermédiaire d'authentification
Langage	TypeScript en mode strict	Typage de bout en bout depuis le contrat OpenAPI
État distant	TanStack Query	Cache, invalidation, réessai, état de chargement
Formulaires	React Hook Form et Zod	Validation locale alignée sur les schémas du backend
Présentation	Tailwind CSS et bibliothèque de composants accessibles	Cohérence visuelle, accessibilité
Tableaux	TanStack Table	Grilles denses, tri, filtrage, virtualisation
Graphiques	Bibliothèque de rendu de graphes React	Tableaux de bord du module de pilotage
Décimaux	Bibliothèque de décimaux	Interdiction du type numérique natif pour les montants
Tests	Vitest et Playwright	Tests unitaires et parcours de bout en bout

7.2 Organisation et principes

```

web/
  app/
    (auth)/          connexion, second facteur, mot de passe
    (centre)/        espace des collaborateurs du Centre
    portefeuille/    pieces/ conformite/ comptabilite/
    obligations/     cloture/ formalites/ pilotage/ referentiel/
    (adherent)/      espace des adherents
    pieces/ echeances/ documents/ factures/
  api/               gestionnaires de routes, intermediaire d'authentification
  lib/
    api/             client genere a partir du contrat OpenAPI
    money.ts auth.ts permissions.ts
  components/
  tests/

```

Les pages de consultation doivent privilégier le rendu côté serveur, réduisant le volume transféré, avantage significatif dans un contexte de bande passante contrainte. Les écrans de saisie et les grilles interactives relèvent du rendu côté client.

Les permissions doivent être appliquées côté serveur, l'interface se contentant de masquer les actions indisponibles. Le masquage relève du confort, jamais de la sécurité.

Les vues destinées aux collaborateurs doivent privilégier la densité et la navigation au clavier, avec raccourcis sur les écrans de saisie répétitive. Les vues destinées aux adhérents doivent être pleinement utilisables sur écran de téléphone, cette exigence conditionnant l'utilité du portail avant la mise à disposition de l'application mobile.

7.3 États de dégradation

L'application doit traiter explicitement l'indisponibilité du réseau et la lenteur, ces situations n'étant pas exceptionnelles. Toute action longue doit présenter un état de progression. Toute soumission doit être protégée contre le double envoi par clé d'idempotence. Toute perte de connexion en cours de saisie doit préserver le contenu du formulaire localement et proposer sa reprise.

8. Application mobile

8.1 Périmètre et pile logicielle

L'application mobile est destinée aux adhérents et couvre trois usages, la capture de pièces justificatives y compris hors ligne, la consultation des échéances et des documents, et le suivi de l'avancement du dossier. Elle ne comporte aucune fonction de gestion comptable, celles-ci demeurant sur le web.

Élément	Retenu	Rôle
Cadre applicatif	Flutter, Dart 3	Base de code unique pour Android et iOS
État applicatif	Riverpod	Injection et état, testable sans interface
Navigation	go_router	Navigation déclarative, liens profonds
Persistance locale	Drift, adossé à SQLite	Base locale typée, file d'envoi, cache de consultation
Réseau	Dio, client généré depuis le contrat	Alignement automatique sur l'interface
Sécurité locale	Magasin sécurisé du système	Conservation des jetons
Capture	Appareil photo, recadrage, compression	Réduction du volume avant envoi
Arrière-plan	Gestionnaire de travaux différés	Synchronisation opportuniste

8.2 Fonctionnement hors ligne

L'application doit être conçue selon le principe du local d'abord. Toute action de l'utilisateur s'applique immédiatement à la base locale et n'attend jamais le réseau. La synchronisation est un processus d'arrière-plan.

Le modèle repose sur une file d'envoi persistée. Chaque action produit une entrée comportant un identifiant généré localement, ordonné dans le temps, qui sert également de clé d'idempotence. Cet identifiant devient l'identifiant définitif de l'entité côté serveur, ce qui évite toute table de correspondance et rend l'opération naturellement rejouable.

```
Table locale file_envoi
  id          identifiant ordonne, genere hors ligne
  type_operation  DEPOT_PIECE | MAJ_METADONNEE | ACCUSE_LECTURE
  charge_utile   donnees serialisees
  fichier_local  chemin du fichier a televerser
  cle_idempotence identique a id
  tentatives     entier
  prochaine_tentative horodatage
  statut         EN_ATTENTE | EN_COURS | ENVOYE | ECHEC_DEFINITIF
  erreur         libelle de la derniere erreur
```

8.2.1 Protocole de synchronisation

La synchronisation comporte deux sens, indépendants et déclenchés séparément.

Dans le sens montant, l'application soumet les entrées de la file par lots ordonnés. Le serveur traite chaque entrée de manière idempotente et retourne, pour chacune, un état de succès, de conflit ou de rejet définitif. Une entrée rejetée définitivement, par exemple pour un document illisible, est retirée de la file et présentée à l'utilisateur. Une entrée en échec transitoire est réessayée selon un délai croissant plafonné.

Dans le sens descendant, l'application demande les modifications postérieures à un jeton de position remis par le serveur lors du dernier échange. Le serveur retourne les entités modifiées et un nouveau jeton. Ce mécanisme est préférable à un filtrage par horodatage, dont la fiabilité dépend de la synchronisation des horloges.

8.2.2 Traitement des conflits

La conception du domaine réduit fortement la surface de conflit. Les pièces justificatives et les factures sont immuables une fois déposées, et ne peuvent donc pas diverger. Les seules données modifiables des deux côtés sont des métadonnées, période de rattachement et qualification. Pour celles-ci, la règle retenue est la primauté de la dernière écriture datée du serveur, toute divergence étant notifiée à l'utilisateur plutôt que silencieusement résolue.

8.3 Contraintes d'exploitation mobile

Les images capturées doivent être compressées localement avant envoi, avec une résolution cible permettant la lecture des mentions d'une facture sans excéder un volume raisonnable. L'envoi doit être fractionné et reprenable, une interruption ne devant pas imposer de recommencer.

La consommation de données doit être visible par l'utilisateur, et une option doit permettre de restreindre la synchronisation des fichiers au réseau sans fil. La base locale ne doit conserver les fichiers déjà transmis que le temps de leur confirmation, afin de contenir l'occupation de l'appareil.

Les versions minimales prises en charge doivent être arrêtées au démarrage de la phase 3 sur la base du parc réel des adhérents, relevé lors de la phase de cadrage, et non sur des hypothèses.

9. Contrats d'interface et génération de clients

La cohérence de l'ensemble repose sur un contrat unique. La chaîne suivante doit être mise en place dès la phase 1 et vérifiée automatiquement.

```
Modeles Pydantic et points d'entree FastAPI
|
v
Contrat OpenAPI 3.1, exporte dans le depot a chaque construction
|
+--> Generation du client TypeScript consomme par Next.js
|
+--> Generation du client Dart consomme par Flutter
```

Le contrat exporté doit être versionné dans le dépôt. L'intégration continue doit échouer lorsque le contrat produit par le code diffère du contrat versionné sans que celui-ci ait été mis à jour, ce contrôle empêchant toute dérive silencieuse entre le backend et ses clients.

Les clients générés ne doivent jamais être modifiés manuellement. Tout comportement additionnel, gestion des jetons, réessai, corrélation, doit être implémenté dans une couche d'enveloppe distincte.

Les types monétaires doivent être configurés explicitement dans les générateurs afin d'être produits en type décimal et non en type numérique natif, cette configuration constituant un point de vigilance à vérifier à chaque montée de version des outils.

10. Stockage documentaire et traitement des pièces

10.1 Stockage

Les fichiers doivent être conservés dans un service de stockage d'objets compatible avec le protocole S3, distinct de la base de données. La base ne conserve que les métadonnées et la référence.

L'adressage doit se faire par le contenu, la clé de l'objet étant dérivée de l'empreinte du fichier. Cette convention procure la déduplication naturelle des pièces identiques et rend toute altération détectable par recalcul.

Les accès en lecture et en écriture depuis les clients doivent s'effectuer par adresses signées de courte durée, délivrées par le backend après contrôle des droits. Les clients ne doivent jamais détenir de justificatif d'accès au stockage.

Une règle de cycle de vie doit assurer la conservation sur dix ans, conformément au droit comptable, avec bascule vers une classe de stockage économique au-delà de la période d'activité courante. Aucune suppression automatique n'est admise avant ce terme.

10.2 Ingestion

Toute pièce déposée doit franchir la séquence suivante avant d'être rendue disponible, à savoir la vérification du type réel par examen de la signature binaire, le contrôle de la taille, l'analyse antivirus, le calcul de l'empreinte, la détection de doublon par comparaison d'empreinte, puis la mise à disposition. Une pièce en échec sur l'une de ces étapes doit être mise en quarantaine et signalée, jamais silencieusement rejetée.

10.3 Reconnaissance de caractères

La reconnaissance doit être atteinte au travers d'une interface abstraite, afin de préserver la liberté de changer de fournisseur. Deux implémentations doivent être livrées.

Implémentation	Usage
Moteur hébergé localement	Traitement au sein du travailleur asynchrone, sans transfert de données à un tiers. Retenu par défaut, la sensibilité des pièces plaidant pour la maîtrise du traitement.
Service d'extraction documentaire externe	Meilleure précision sur les documents structurés. Activable par paramètre, sous réserve d'une décision expresse du maître d'ouvrage au regard de la confidentialité.

L'extraction doit produire, pour chaque champ, une valeur et un indice de confiance. Un seuil paramétrable détermine la présentation à l'utilisateur, les champs sous le seuil étant signalés pour vérification. La validation demeure humaine en toute hypothèse, l'extraction ne constituant qu'une assistance à la saisie.

11. Intégrations externes

11.1 Principe d'abstraction

Chaque système tiers doit être atteint au travers d'une interface définie par le domaine, dont les implémentations sont interchangeables. Chaque interface doit disposer d'une implémentation de substitution manuelle, permettant à la plateforme de fonctionner en l'absence de service automatisé.

```
class VerificationIdentifiantFiscal(Protocol):
    def verifier(self, niu: str, a_la_date: date) -> ResultatVerification: ...

# Implementations livrees
# AdaptateurPortailFiscal    appel au service publie, si disponible
# AdaptateurCacheLocal      reponse depuis le referentiel des tiers connus
# AdaptateurSaisieManuelle  attestation saisie et justifiee par un collaborateur
```

11.2 Intégrations prévues

Interface	État initial	Prescriptions
Vérification d'identifiant fiscal	Cache et saisie manuelle	Cache d'une durée paramétrable, indication explicite de la source et de la fraîcheur du résultat dans le rapport de conformité
Télédéclaration	Production de fichiers et dépôt manuel	Génération aux formats attendus, enregistrement de l'accusé et de la quittance, conception permettant l'automatisation ultérieure sans reprise du domaine
Facturation électronique	Modèle de données anticipé	Champs d'identifiant de transmission, d'horodatage, de statut de validation et d'empreinte prévus dès l'origine
Relevés bancaires	Import de fichiers	Prise en charge des formats usuels, détection de doublon, réconciliation assistée
Monnaie électronique	Import de fichiers	Normalisation des libellés, rapprochement avec les pièces
Courriel	Fournisseur d'acheminement	Suivi de remise, gestion des retours
Message court	Agrégateur local	Segmentation, suivi de remise, plafond de coût paramétrable
Messagerie instantanée	Interface professionnelle	Réception de pièces et notifications, sous réserve de faisabilité confirmée en conception

Tout appel sortant doit être protégé par un délai maximal, un nombre de tentatives borné et un disjoncteur ouvrant le circuit après un seuil d'échecs consécutifs. L'indisponibilité d'un tiers ne doit jamais empêcher le travail du collaborateur, mais se traduire par un état explicite et une reprise différée.

12. Exploitation

12.1 Environnements

Environnement	Données	Usage
Développement	Jeu de données synthétique	Poste des développeurs, exécution par conteneurs
Recette	Données réelles anonymisées	Recette du maître d'ouvrage, essais du fiscaliste
Production	Données réelles	Exploitation

Aucune donnée réelle non anonymisée ne doit être présente en développement ou en recette. Une procédure d'anonymisation reproductible doit être livrée, conservant les caractéristiques statistiques nécessaires aux tests sans permettre l'identification.

12.2 Hébergement

Deux options doivent être présentées au maître d'ouvrage, la décision lui appartenant, conformément au document fonctionnel.

Option	Caractéristiques
Infrastructure en nuage	Base gérée avec sauvegarde et restauration à un instant donné, stockage d'objets géré, conteneurs applicatifs. Disponibilité et exploitation supérieures, coût récurrent, dépendance à un fournisseur et question de la localisation des données.
Infrastructure locale	Ensemble déployé par conteneurs sur un serveur maîtrisé, stockage d'objets auto-hébergé. Maîtrise complète et localisation garantie, mais responsabilité entière de l'exploitation, de la sauvegarde et de la continuité électrique.

Quelle que soit l'option, le déploiement doit être décrit par des fichiers versionnés et reproductible sans intervention manuelle.

12.3 Sauvegarde et continuité

Élément	Prescription
Sauvegarde de la base	Sauvegarde complète quotidienne et archivage continu des journaux de transaction, permettant une restauration à un instant donné
Sauvegarde du stockage	Versionnement des objets et réplication vers un emplacement distinct
Chiffrement	Sauvegardes chiffrées, clés conservées séparément
Rétention	Trente jours en quotidien, douze mois en mensuel
Test de restauration	Restauration complète en environnement isolé au moins deux fois par an, procès-verbal remis au maître d'ouvrage

Objectifs de reprise	Perte de données maximale admissible d'une heure, délai de remise en service de quatre heures ouvrées, à confirmer par le maître d'ouvrage
----------------------	--

12.4 Supervision

La journalisation doit être structurée et exploitable par recherche, chaque entrée portant l'identifiant de corrélation, l'identifiant de locataire et l'identifiant d'utilisateur. Aucune donnée personnelle ni aucun secret ne doit apparaître dans les journaux.

Les métriques techniques et métier doivent être exposées, notamment le temps de réponse par point d'entrée, le taux d'erreur, la profondeur des files de travaux, la durée d'évaluation du moteur de conformité et le nombre d'obligations en retard.

Les erreurs applicatives doivent être collectées et notifiées. Des alertes doivent être définies au minimum sur l'indisponibilité du service, la saturation d'une file, l'échec d'une sauvegarde, la rupture de la chaîne d'audit et l'approche d'une échéance déclarative sur un dossier incomplet.

13. Qualité et intégration continue

13.1 Organisation des dépôts

Un dépôt unique regroupant les trois bases de code doit être retenu, avec les répertoires `backend`, `web`, `mobile`, `contrats` et `infra`. Cette organisation permet à une modification d'interface et à ses conséquences sur les clients d'être portées par une seule proposition de fusion, ce qui est déterminant pour maintenir la cohérence du contrat.

13.2 Discipline de contribution

La branche principale doit être protégée, toute modification passant par une proposition de fusion soumise à revue et à la réussite de l'ensemble des contrôles automatiques. Les messages de validation doivent suivre une convention permettant la génération automatique du journal des versions. Toute modification touchant une règle de conformité, un calcul fiscal ou le schéma de données doit faire l'objet d'une revue explicitement mentionnée.

13.3 Chaîne d'intégration

- | | |
|---|--------------------------------|
| 1. Analyse statique et mise en forme | ruff, eslint, dart analyze |
| 2. Vérification de typage | mypy strict, tsc, dart |
| 3. Tests unitaires et de domaine | pytest, vitest, flutter test |
| 4. Tests d'intégration sur base réelle | conteneurs éphémères |
| 5. Vérification des migrations | application et retour arrière |
| 6. Exportation et comparaison du contrat | échec si dérive non versionnée |
| 7. Régénération et compilation des clients | TypeScript et Dart |
| 8. Analyse de vulnérabilités des dépendances | |
| 9. Contrôles spécifiques du projet | point 13.4 |
| 10. Construction des images et publication | |
| 11. Déploiement en recette, puis en production après validation | |

13.4 Contrôles spécifiques automatisés

Cinq contrôles propres au projet doivent être implémentés comme des tests bloquants. Ils traduisent en vérifications automatiques les critères d'acceptation du document fonctionnel.

Contrôle	Objet
Absence de valeur légale codée	Détection de littéraux numériques suspects dans les modules métier et vérification que tout taux ou seuil provient du service de paramètres
Interdiction du flottant	Rejet de toute déclaration de montant en type flottant dans les modèles, schémas et clients générés
Isolation du domaine	Vérification qu'aucun module de domaine n'importe d'infrastructure et qu'aucune dépendance circulaire n'existe entre modules
Cloisonnement effectif	Test posant deux locataires et vérifiant qu'aucune requête ne franchit la frontière, y compris en cas d'erreur applicative
Immuabilité	Test tentant la modification et la suppression d'une écriture validée et vérifiant le rejet par la base

13.5 Stratégie de tests

Niveau	Couverture visée	Portée
Domaine et calculs	Quatre-vingt-dix pour cent	Règles de gestion, calculs fiscaux, arrondis, périodes
Règles de conformité	Intégralité	Deux cas au minimum par règle, exécutés à chaque intégration et à chaque activation
Intégration	Chemins nominaux et erreurs	Points d'entrée sur base réelle éphémère, transactions et cloisonnement
Bout en bout	Onze parcours	Un parcours par cas d'utilisation du document fonctionnel
Non-régression comptable	Jeux de référence	Comparaison des états et de la liasse produits à des résultats de référence validés par le référent métier
Charge	Objectifs du chapitre 15	Édition de balance annuelle, contrôle de masse, échéancier consolidé

Les tests de non-régression comptable revêtent une importance particulière. Une balance et une liasse validées par le référent métier doivent être figées comme références, et toute évolution du code produisant un écart doit être expressément justifiée avant d'être acceptée.

14. Performance et dimensionnement

14.1 Objectifs

Opération	Objectif	Mesure
Affichage d'une page de consultation	Moins de 3 secondes	Neuvième décile, en conditions de connexion mobile courante
Contrôle de conformité d'une facture	Moins de 2 secondes	Neuvième décile, hors vérification externe
Édition d'une balance annuelle	Moins de 10 secondes	Sur un dossier de volume représentatif
Réponse de l'interface de programmation	Moins de 300 millisecondes	Médiane, hors traitements différés
Contrôle de masse mensuel	Moins de 30 minutes	Sur l'ensemble du portefeuille

14.2 Hypothèses de volumétrie

Le dimensionnement initial repose sur les hypothèses suivantes, à confirmer lors de la phase de cadrage puisqu'elles figurent parmi les points restés ouverts dans le document fonctionnel.

Grandeur	Hypothèse de travail
Entreprises suivies à trois ans	Quelques centaines
Pièces justificatives par entreprise et par an	De cent à mille selon le régime
Lignes d'écriture par entreprise et par an	De quelques centaines à quelques dizaines de milliers
Utilisateurs simultanés du Centre	Quelques dizaines
Volume documentaire annuel	Quelques centaines de gigaoctets

Ces ordres de grandeur ne présentent aucune difficulté pour la pile retenue, à condition que l'indexation décrite au point 5.9 soit respectée et que les états comptables soient produits par agrégation en base plutôt qu'en mémoire applicative.

14.3 Règles de conception liées à la performance

Les états volumineux doivent être produits par des requêtes d'agrégation exécutées en base, jamais par chargement des lignes en mémoire. Les listes doivent être paginées par curseur. Les exports doivent être produits en flux et non constitués intégralement en mémoire. Les requêtes en cascade sur une collection doivent être proscrites, un test dédié détectant leur apparition. Le cache des paramètres du référentiel doit être partagé et invalidé à l'écriture.

15. Décisions d'architecture

Chaque décision structurante doit faire l'objet d'une fiche versionnée dans le dépôt, énonçant le contexte, la décision, les alternatives examinées et les conséquences. Les décisions initiales sont récapitulées ci-après.

Réf.	Décision	Conséquence principale
D01	Monolithe modulaire plutôt que services indépendants	Transactions comptables atomiques, exploitation simplifiée, discipline de découpage à maintenir par test d'architecture
D02	Exécution synchrone du backend	Transactions maîtrisées, appels externes reportés sur les travailleurs
D03	Cloisonnement par sécurité au niveau des lignes	Isolation garantie par la base, obligation de poser le contexte à chaque transaction
D04	Référentiel temporel avec contrainte d'exclusion	Impossibilité structurelle de définir deux valeurs concurrentes à une même date
D05	Immuabilité imposée par déclencheur	Correction par contre-passation uniquement, y compris pour un accès direct à la base
D06	Journal d'audit chaîné par empreinte	Altération détectable, vérification quotidienne à exploiter
D07	Compteurs verrouillés pour la numérotation	Continuité garantie, sérialisation limitée à une ligne par entreprise et exercice
D08	Moteur de règles dédié à langage restreint	Sûreté, déterminisme et explicabilité, au prix du développement de l'évaluateur
D09	Contrat OpenAPI comme source unique	Clients générés, dérive détectée automatiquement
D10	Interdiction absolue du flottant pour les montants	Contrôle automatisé bloquant, configuration explicite des générateurs de clients
D11	Mobile local d'abord avec file d'envoi idempotente	Fonctionnement sans réseau, identifiants générés côté client
D12	Dépôt unique regroupant les trois bases de code	Cohérence du contrat préservée, chaîne d'intégration plus longue

16. Risques techniques

Risque	Gravité	Traitement
Erreur de représentation ou d'arrondi des montants	Élevée	Interdiction du flottant contrôlée automatiquement, règles d'arrondi paramétrées, tests de non-régression sur états de référence
Fuite de données entre locataires ou entre portefeuilles	Élevée	Cloisonnement au niveau de la base, test dédié bloquant, revue de sécurité avant mise en production
Défaut de conception du moteur de règles compromettant l'auditabilité	Élevée	Fondement légal obligatoire, projection conservée, cas de test par règle, état d'essai avant activation
Charge excessive liée à trois bases de code pour deux personnes	Élevée	Report de l'application mobile en phase 3, portail web pleinement utilisable sur mobile dès la phase 2
Dérive entre le backend et ses clients	Moyenne	Contrat versionné, comparaison bloquante en intégration continue, dépôt unique
Complexité de la synchronisation hors ligne	Moyenne	Domaine conçu pour l'immutabilité, identifiants générés côté client, idempotence systématique
Indisponibilité prolongée d'un service tiers	Moyenne	Ports abstraits, mode manuel de substitution, disjoncteur et reprise différée
Absence de reprise possible par un tiers	Moyenne	Technologies courantes, documentation d'architecture, fiches de décision, code remis intégralement
Perte de données par défaut de sauvegarde	Élevée	Archivage continu des journaux, test de restauration semestriel avec procès-verbal

17. Validation

Le présent cahier des charges technique complète le cahier des charges fonctionnel et n'en modifie ni le périmètre ni les engagements. Il est soumis au maître d'ouvrage pour information et pour validation des seuls éléments appelant sa décision, à savoir l'option d'hébergement retenue au point 12.2, les objectifs de reprise énoncés au point 12.3, le recours éventuel à un service d'extraction documentaire externe mentionné au point 10.3, et le report de l'application mobile en phase 3 recommandé au point 1.2.

Les autres dispositions relèvent de la responsabilité de la maîtrise d'œuvre et constituent la référence normative des développements, opposable en revue de code et lors de la recette technique.

Fait à Douala, en deux exemplaires originaux, le

POUR LE MAÎTRE D'OUVRAGE Broad Range Consulting Group Madame HIMSTA Paule Diane Précédée de la mention « lu et approuvé »	POUR LA MAÎTRISE D'ŒUVRE Monsieur TCHAMBA TCHAKOUNTE Edwin Précédée de la mention « lu et approuvé »
Cachet	POUR LA MAÎTRISE D'ŒUVRE Monsieur TONBA DJIMGOU Loïc Précédée de la mention « lu et approuvé »

Annexe. Correspondance entre cas d'utilisation et composants

Cas	Intitulé	Module backend	Composants sollicités
UC01	Instruire un dossier de création d'entreprise	formalites	Web, stockage documentaire, moteur de règles jeu formalités, notifications
UC02	Enrôler un nouvel adhérent	portefeuille	Web, référentiel temporel, module obligations pour la génération de l'échéancier
UC03	Déposer une pièce justificative	pieces	Mobile hors ligne, web, stockage, travailleur de reconnaissance, moteur de règles
UC04	Relancer les pièces manquantes	pieces	Ordonnanceur, rapprochement des mouvements, notifications multicanal
UC05	Contrôler la conformité d'une facture	conformite	Moteur de règles, référentiel temporel, port de vérification d'identifiant, cache
UC06	Émettre une facture conforme	conformite et comptabilite	Web, compteur de numérotation verrouillé, moteur en mode bloquant, génération du document
UC07	Comptabiliser une pièce justificative	comptabilite	Web, plan de comptes dérivé, déclencheur d'immuabilité, report des attributs fiscaux
UC08	Générer et suivre l'échéancier	obligations	Référentiel des types d'obligation, ordonnanceur, notifications
UC09	Préparer et déposer une déclaration	obligations	Agrégation en base, exclusions issues de la conformité, port de télédéclaration
UC10	Clôturer l'exercice et produire la liasse	cloture	Correspondance comptes et postes, tableau de passage alimenté par la conformité, contrôles de cohérence, export
UC11	Administrer le référentiel et les règles	referentiel	Web, contrainte d'exclusion temporelle, état d'essai, cas de test, journal d'audit